

Advanced Machine Learning

DATA 642

Lecture 13 — Hyperparameters

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Number of Hidden Layers

- For many problems you can start with just one or two hidden layers and it will work just fine (e.g., you can easily reach above 97% accuracy on the MNIST dataset using just one hidden layer with a few hundred neurons, and above 98% accuracy using two hidden layers with the same total amount of neurons, in roughly the same amount of training time).
- For more complex problems, you can gradually ramp up the number of hidden layers, until you start over-fitting the training set. Very complex tasks, such as large image classification or speech recognition, typically require networks with dozens of layers (or even hundreds, but not fully connected ones), and they need a huge amount of training data.
 - No need to train such networks from scratch: it is much more common to reuse parts of a pre-trained state-of-the-art network that performs a similar task. Training will be a lot faster and require much less data.

Number of Neurons per Hidden Layer

- The number of neurons in the input and output layers is determined by the type of input and output your task requires. For example, the MNIST task requires 784 input neurons and 10 output neurons.
- For the hidden layers, it used to be a common practice to size them to form a pyramid, with fewer and fewer neurons at each layer—the rationale being that many low-level features can coalesce into far fewer high-level features. (However, this practice is not very popular anymore)
- Just like for the number of layers, you can try increasing the number of neurons gradually until the network starts over-fitting. In general you will get more bang for the buck by increasing the number of layers than the number of neurons per layer. Unfortunately, as you can see, finding the perfect amount of neurons is still somewhat of a dark art.
- A simpler approach is to pick a model with more layers and neurons than you actually need, then use early stopping to prevent it from over-fitting (and other regularization techniques, such as dropout). This has been dubbed the "stretch pants" approach: instead of wasting time looking for pants that perfectly match your size, just use large stretch pants that will shrink down to the right size.

Learning Rate, Batch Size and Other Hyperparameters

- The learning rate stands as a crucial hyperparameter, often dictating training success. Typically, the optimal learning rate is approximately half of the maximum rate, beyond which training diverges. A straightforward tuning method involves starting with a large value, causing divergence, then iteratively dividing by 3 until divergence ceases. This process often leads to the optimal learning rate.
- Choosing a better optimizer than plain old Mini-batch Gradient Descent (and tuning its hyperparameters) is also quite important.
- The batch size significantly influences model performance and training time. Optimal batch sizes are typically smaller than 32. While larger batches provide more precise gradient estimates, the optimization landscape's complexity renders this advantage negligible. However, batch sizes greater than 10 leverage hardware optimizations, especially for matrix multiplications, speeding up training. Additionally, Batch Normalization benefits from batch sizes not smaller than 20.
- The choice of the activation is also important: in general, the ReLU activation function will be a good default for all hidden layers. For the output layer, it really depends on your task.
- In most cases, the number of training iterations does not actually need to be tweaked: just use early stopping instead.

Hyperparameter Tuning

How do you know what combination of hyperparameters is the best for your task?

One option is to simply try many combinations of hyperparameters and see which one works best on the validation set (or using K-fold cross-validation).

- Using randomized search is not too hard, and it works well for many fairly simple problems. However, when training is slow (e.g., for more complex problems with larger datasets), this approach will only explore a tiny portion of the hyperparameter space. You can partially alleviate this problem by assisting the search process manually:
 - First run a quick random search using wide ranges of hyperparameter values,
 - then run another search using smaller ranges of values centered on the best ones found during the first run, and so on.

Other options for hyper-parameter tuning

- Here are a few Python libraries you can use to optimize hyperparameters:
 - **Hyperopt**: a popular Python library for optimizing over all sorts of complex search spaces (including real values such as the learning rate, or discrete values such as the number of layers).
 - **Hyperas**, **kopt** or **Talos**: optimizing hyperparameters for Keras model (the first two are based on Hyperopt).
 - **Scikit-Optimize (skopt)**: a general-purpose optimization library. The `Bayes SearchCV` class performs Bayesian optimization using an interface similar to `Grid SearchCV`.
 - **Spearmint**: a Bayesian optimization library.
 - **Sklearn-Deap**: a hyperparameter optimization library based on evolutionary algorithms, also with a GridSearchCV-like interface.